

基于噪声监测数据的城市声环境治理平台

数据库系统原理课程设计报告



学 号 _____

姓 名 _____

专 业 _____ 计算机科学与技术

授课老师 _____ 李文根

目录

基于噪声监测数据的城市声环境治理平台	1
一. 概述	5
1.1 课题背景	5
1.2 编写目的	5
二. 需求分析	5
2.1 功能需求	5
2.1.1 用户与权限管理	6
2.1.2 基础数据管理	6
2.1.3 噪声数据接入与存储	6
2.1.4 告警管理与处置	7
2.1.5 统计分析与可视化	7
2.2 非功能需求	7
2.2.1 性能需求	7
2.2.2 可靠性与可用性	7
2.2.3 安全性需求	8
2.2.4 可维护性与扩展性	8
2.3 数据字典	8
2.4 数据流图	10
三. 可行性分析	11
3.1 技术可行性	11
3.2 应用可行性	11
四. 概念设计	12
4.1 实体	12
4.2 实体属性局部 E-R 图	15
4.3 全局 E-R 图	18
五. 逻辑设计	18
5.1 E-R 图向关系模型的转变	18
5.2 数据模型的优化及规范化设计	19
5.3 表结构设计	19
5.4 物理设计	22
5.4.1 存储结构与分区策略	23
5.4.2 索引设计	23
5.4.3 视图与预计算	24
5.4.4 触发器与自动化	24
5.4.5 空间数据存储	25
六. 项目管理	25
6.1 框架选择	25
6.2 开发平台	25
七. 系统实现	25
7.1 系统架构搭建	25
7.2 系统逻辑设计	26
7.3 具体功能编写	26

7.4 功能测试.....	27
八. 总结.....	37
参考文献.....	38

摘要 随着城市化进程的加速,环境噪声污染已成为影响居民生活质量的重要问题,传统的噪声监管手段存在时效性差、数据孤岛严重及人工成本高等痛点。针对上述问题,本文设计并实现了一套基于物联网与时序数据库技术的“城市噪声在线监测系统”。该系统采用 B/S 架构,后端基于 Python FastAPI 框架构建,前端采用 Vue.js 3 开发。在数据存储层,系统创新性地引入 PostgreSQL 结合 TimescaleDB 时序数据库引擎,利用超表(Hypertable)技术与列式压缩机制,有效解决了海量高频噪声数据的存储与查询性能瓶颈;同时集成 PostGIS 空间数据库引擎,实现了监测点位的空间管理与可视化。系统核心功能涵盖了设备数据的实时采集与清洗、基于数据库触发器的超标自动告警、利用物化视图实现的连续聚合统计分析,以及基于 RBAC 模型的用户权限管理。测试结果表明,该系统能够稳定处理高并发设备上报请求,实现毫秒级的查询响应与秒级的告警推送,为城市环境噪声的精细化治理提供了强有力的数据支撑与技术保障。

关键词: 城市噪声监测; 时序数据库; TimescaleDB; PostGIS; FastAPI; Vue.js

一. 概述

1.1 课题背景

随着我国城市化进程的不断推进，城市噪声污染问题日益凸显，已成为继大气污染、水污染之后的第三大环境问题。近年来，交通噪声、建筑施工噪声、社会生活噪声等各类噪声源持续增加，多种噪声源交织存在，使得城市声环境呈现出复杂多变的特点，严重影响居民生活质量和身心健康。随着环境监测技术的快速发展，各城市已逐步建立起覆盖重点区域的噪声自动监测网络，产生了海量的噪声监测数据。然而，目前许多城市的噪声监测工作仍面临数据分散管理、时空维度缺失、时序查询低效、告警机制滞后以及分析手段单一等现实困境，导致治理措施缺乏针对性和时效性，亟需建立专业的数据库系统对噪声监测数据进行规范化管理。在此背景下，为提升城市环境管理水平，本次数据库应用系统开发搭建了一个基于 PostgreSQL、TimescaleDB 与 PostGIS 的城市声环境治理平台。

1.2 编写目的

本系统是一个面向城市环境监管部门的噪声监测数据管理与治理平台，采用 FastAPI 与 Vue.js 前后端分离架构，利用 TimescaleDB 扩展实现时序数据的高效存储与查询，利用 PostGIS 扩展提供地理空间分析能力。系统通过 Docker 容器化部署，支持设备实时上报与 CSV/JSON 批量导入，实现了从数据采集、存储、分析到可视化展示的全链条管理。

本平台旨在通过对噪声监测数据的集中存储、高效查询、统计分析和可视化展示，为环保部门及相关机构提供科学决策的数据支撑。具体而言，项目致力于建立统一的数据存储体系，构建规范化的数据库模型以实现多维数据的关联管理；提升时序数据处理能力，利用 TimescaleDB 超表技术保障亿级数据下的查询性能；强化空间分析功能，基于 PostGIS 提供地理围栏查询与区域噪声热力图展示；实现自动化告警机制，通过数据库触发器避免人工漏报；支撑多角度数据分析，提供按时间、空间及状态的多维度统计与可视化；并保障系统的安全性及可追溯性，实现基于角色的权限控制与操作审计。通过上述功能的实现，本系统将推动城市噪声监管由“经验驱动”向“数据驱动”转变，提高治理效率与科学化水平，同时为未来智慧城市中环境数据管理系统的建设提供可借鉴的技术路径和实践基础，具有较强的现实意义与应用价值。

二. 需求分析

2.1 功能需求

本系统旨在为城市环境监管部门提供全方位的噪声监测与治理能力，核心功能需求如下：

2.1.1 用户与权限管理

系统需构建完善的身份认证与授权体系，支持多级用户角色，实现基于角色的访问控制（RBAC），确保数据安全与操作合规。首先，系统应提供用户注册与登录功能，允许用户提交用户名、密码、邮箱及手机号进行账号申请，并提供安全的登录认证机制，支持用户名/密码登录，登录成功后签发 JWT 令牌用于后续会话管理，同时支持用户修改个人密码及基本信息。

其次，系统需实现精细的角色与权限控制，具体内容如下。

超级管理员拥有系统最高权限，负责用户管理（包括创建、编辑、禁用/启用用户账号，重置用户密码及分配角色）、基础数据配置（区域、监测点、设备的增删改查）、全局数据监控以及高级告警处置（如删除误报数据与归档历史告警）。

区域运维员负责特定责任区域的日常运维，包括接收并处置责任区域内的噪声超标告警并填写处置记录、调整责任区域内监测点的昼/夜噪声报警阈值、查看区域报表（历史趋势、超标排名及合规率分析）以及上报设备故障。

普通用户仅具备查看权限，可浏览实时监测地图以查看点位分布与实时噪声等级、查阅公开的噪声统计报表与排名信息，并维护个人账户信息。

2.1.2 基础数据管理

系统需提供对核心业务实体的全生命周期管理功能，确保监测网络基础数据的准确性与完整性。在区域管理方面，系统应支持创建、编辑和删除功能区域，维护区域名称、区域类型（如居民区、商业区、工业区等）、中心点经纬度坐标及地理边界范围，并支持按区域类型进行筛选与查询。在监测点管理方面，系统应支持监测点的全生命周期管理（部署、维护、撤销），维护点位名称、所属区域、经纬度坐标、默认昼/夜阈值及当前状态（正常/维护/停用），并自动维护监测点的空间索引以支持高效的地图查询。在设备管理方面，系统应建立设备台账，管理监测设备的硬件信息，包括设备序列号（唯一标识）、设备型号、生产厂商、安装时间、当前在线状态及电量信息，并支持设备与监测点的绑定与解绑操作，确保数据来源可追溯。

2.1.3 噪声数据接入与存储

系统需具备高并发、多模式的数据接入能力，并保障海量数据的高效存储与全生命周期管理。针对实时数据上报，系统应提供标准的 RESTful API 接口，支持监测设备或网关实时推送包含设备序列号、采集时间戳、噪声分贝值、温湿度及电池电量百分比的数据，并对上报数据进行实时校验，过滤非法格式或超出合理范围的异常数据。针对历史数据批量导入，系统应支持管理员上传 CSV 或 JSON 格式的历史数据文件进行批量导入，提供导入任务管理功能以记录任务状态、成功/失败记录数及错误日志，并具备去重机制。在时序数据存储优化方面，系统应利用 TimescaleDB 超表技术存储噪声监测数据，实现自动分块策略（如

按 7 天分块）以优化查询性能，配置自动压缩策略（如 30 天前数据自动压缩）以降低存储成本，并设置数据保留策略（如保留 24 个月）以自动清理过期数据。

2.1.4 告警管理与处置

系统需实现从告警生成、分发、处置到归档闭环的全流程管理，确保噪声超标事件得到及时响应。首先是自动告警生成，系统应基于数据库触发器实现实时的超标判断逻辑，当监测数据入库时自动对比该点位的当前生效阈值（区分昼/夜），若数据超标则自动生成告警记录并判定告警等级（低/中/高/紧急）。其次是多维度告警查询，系统应提供告警列表视图，支持按时间范围、区域类型、告警等级及处理状态等多维度筛选，并支持模糊搜索。最后是告警处置闭环，运维人员可对“待处理”告警进行确认、标记已解决或标记误报（忽略）等处置操作，处置时需填写备注信息，系统应自动记录处置人、处置时间及处置动作类型，形成完整的处置日志。

2.1.5 统计分析与可视化

系统需提供直观的数据展示与深度分析功能，将海量监测数据转化为辅助科学决策的图表与报表。实时监测地图应基于 GIS 地图展示全市监测点的分布情况，图标根据实时状态动态着色，点击可查看详细信息，并提供时间轴回放功能以展示过去 24 小时的噪声状态变化。综合仪表盘应提供全局概览页面，展示当日平均噪声分贝、监测数据总量、超标率及待处理告警数等关键指标。趋势分析图表应提供小时级和日级噪声变化趋势折线图，支持多点位或多区域的对比分析。区域排名与合规分析应自动统计并展示噪声超标次数最多的区域 TOP5 排名，并提供区域噪声合规率饼图，展示达标数据与超标数据的比例分布，辅助识别重点治理区域。

2.2 非功能需求

2.2.1 性能需求

高并发写入：系统需支持数百台设备同时在线上报，峰值写入速度不低于 5000 条/秒。

低延迟查询：实时监测数据与最近 24 小时趋势查询的响应时间应控制在 500ms 以内。

海量数据支撑：在亿级数据规模下，统计分析报表的生成时间应控制在 3 秒以内。

2.2.2 可靠性与可用性

数据一致性：通过数据库事务与外键约束，确保业务数据的完整性与一致性。

服务高可用：后端服务与数据库均支持容器化部署，具备故障自动重启能力。

数据备份：支持定时全量备份与增量备份，确保数据在灾难情况下的可恢复性。

2.2.3 安全性需求

身份认证：采用 JWT Token 机制进行用户身份校验，防止未授权访问。

权限隔离：严格执行 RBAC 策略，确保用户仅能访问其权限范围内的数据。

审计追踪：关键业务操作（如删除数据、处置告警）需全程记录审计日志，支持事后追溯。

2.2.4 可维护性与扩展性

模块化设计：采用前后端分离架构，各功能模块耦合度低，便于独立维护与升级。

水平扩展：数据库层利用 TimescaleDB 的分块特性，支持存储容量的线性扩展。

2.3 数据字典

1. user_id, username, password_hash, role_id, status

描述: 用户认证模块使用的核心用户数据字段，包括：

user_id（用户 ID）：用户唯一标识，系统自动生成的主键。

username（用户名）：用户登录账号，系统内唯一。

password_hash（密码哈希）：经 bcrypt 加密存储的用户密码。

role_id（角色 ID）：关联角色表，定义用户权限级别（super_admin/area_operator/public_user）。

status（状态）：账号状态标识（active/inactive/locked/pending）。

来源: 用户注册、管理员创建

数据类型: 整数、字符串、枚举

用途: 被认证模块用于用户登录验证、JWT 令牌生成、权限校验，控制用户对系统各功能模块的访问权限。

2. point_id, point_name, latitude, longitude, threshold_db, area_id

描述: 监测点管理模块使用的核心点位数据字段，包括：

point_id（监测点 ID）：监测点唯一标识，主键。

point_name（监测点名称）：监测点位置描述。

latitude（纬度）：WGS84 地理纬度坐标，范围-90~90。

longitude（经度）：WGS84 地理经度坐标，范围-180~180。

threshold_db（噪声阈值）：该点位的噪声超标判定阈值，单位分贝（dB），默认 65.00。

area_id（区域 ID）：关联所属监测区域，外键。

来源: 运维人员配置、系统初始化导入

数据类型: 整数、字符串、地理坐标（DECIMAL）、数值

用途: 被地图展示模块用于点位渲染，被数据处理模块用于噪声超标判定，被统计模块用于区域聚合分析。

3. point_id, measured_at, db_value, temperature, humidity, is_exceed

描述: 噪声数据采集模块使用的核心监测数据字段, 包括:

point_id (监测点 ID): 数据所属监测点, 外键。

measured_at (测量时间): 噪声数据采集的时间戳, TimescaleDB 分区键。

db_value (噪声值): 等效声级测量值, 单位分贝 (dB), 范围 0~200。

temperature (温度): 环境温度, 单位摄氏度 (°C)。

humidity (湿度): 环境相对湿度, 单位百分比 (%)。

is_exceed (超标标志): 是否超过阈值, 由数据库触发器自动计算。

来源: 监测设备实时上报、CSV 批量导入

数据类型: 整数、时间戳 (TIMESTAMPTZ)、数值 (DECIMAL)、布尔值

用途: 被数据分析模块用于计算小时/日统计 (平均值、最大值、最小值、超标率), 被报警模块用于触发超标告警, 被前端图表模块用于趋势展示。

4. alert_id, point_id, triggered_at, db_value, threshold_db, level, status

描述: 报警管理模块使用的核心告警数据字段, 包括:

alert_id (报警 ID): 报警记录唯一标识, 主键。

point_id (监测点 ID): 触发报警的监测点, 外键。

triggered_at (触发时间): 报警生成的时间戳。

db_value (噪声值): 触发报警时的实际噪声测量值。

threshold_db (阈值): 触发报警时的超标判定阈值。

level (报警级别): 超标严重程度 (minor: <10dB 超标/moderate: 10-20dB/severe: >20dB)。

status (报警状态): 处理状态 (open 待处理/resolved 已处置/archived 已归档)。

来源: 噪声数据写入时由数据库触发器自动生成

数据类型: 整数、时间戳、数值、枚举字符串

用途: 被报警列表页面用于展示待处理告警, 被运维人员用于报警处置流程, 被统计模块用于计算超标点位数和告警趋势。

5. point_id, day, samples, avg_db, min_db, max_db, exceed_count

描述: 统计分析模块使用的日聚合统计数据字段, 包括:

point_id (监测点 ID): 统计所属监测点。

day (统计日期): 按天聚合的时间桶 (time_bucket)。

samples (采样数): 当日该点位的数据采集条数。

avg_db (平均噪声): 当日平均等效声级。

min_db (最小噪声): 当日最低噪声值。

max_db (最大噪声): 当日最高噪声值。

exceed_count (超标次数): 当日超标数据条数。

来源: TimescaleDB 连续聚合视图 (mv_daily_point_stats) 自动刷新

数据类型: 整数、日期、数值
用途: 被仪表盘概览模块用于展示关键指标, 被区域排名模块用于计算超标重灾区, 被报表导出模块用于生成日/月统计报表。

6. area_id, area_name, area_type, latitude, longitude

描述: 区域管理模块使用的核心区域数据字段, 包括:

- area_id (区域 ID): 区域唯一标识, 主键。
- area_name (区域名称): 监测区域名称。
- area_type (区域类型): 功能区分类 (residential /commercial /industrial /mixed)。
- latitude (纬度): 区域中心点纬度坐标。
- longitude (经度): 区域中心点经度坐标。

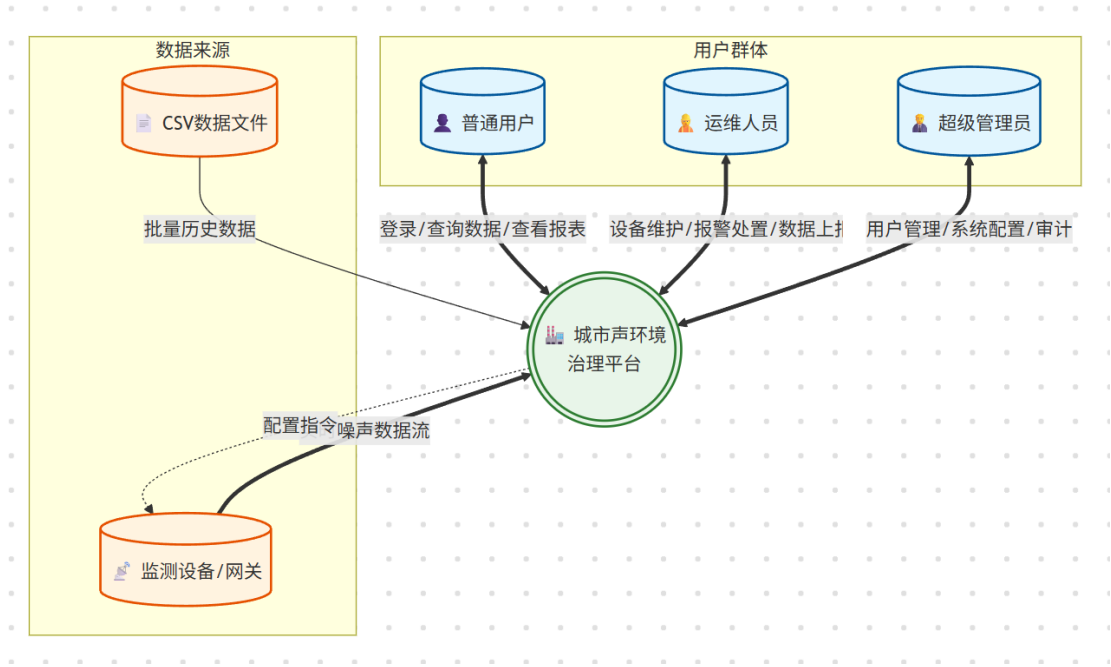
来源: 管理员配置

数据类型: 整数、字符串、枚举、地理坐标

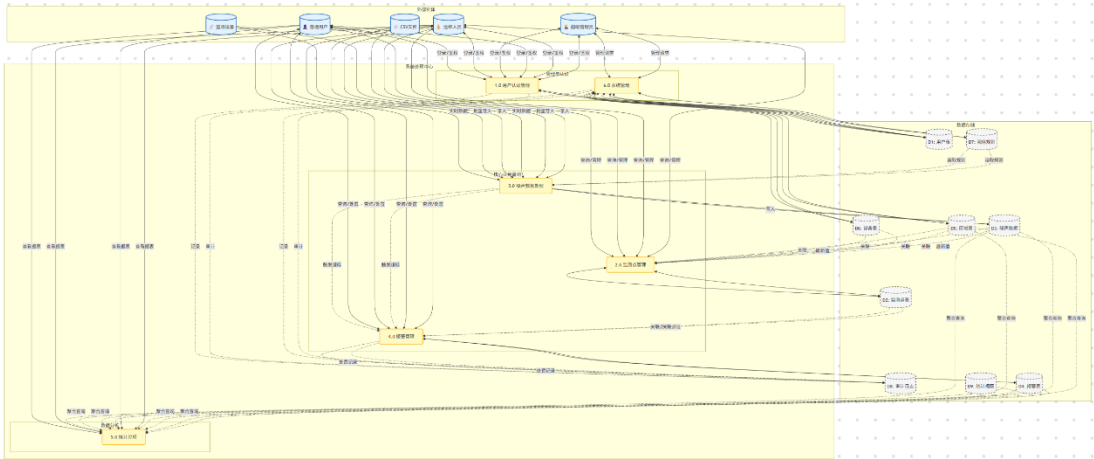
用途: 被监测点管理模块用于点位归属分类, 被统计模块用于按区域聚合分析, 被地图模块用于区域边界展示和筛选。

2.4 数据流图

1、顶层数据流图



2、0 层数据流图



三. 可行性分析

3.1 技术可行性

本项目在技术选型上充分考虑了系统的性能、扩展性与开发效率，采用了一套成熟且先进的前后端分离架构。在数据存储层，针对物联网场景下海量噪声数据的写入与查询挑战，项目选用了 PostgreSQL 结合 TimescaleDB 时序数据库技术。利用其核心的“超表”技术，系统将数据按时间自动分片，在保证标准 SQL 兼容性的同时，实现了每秒数万条记录的高吞吐量写入；同时利用连续聚合功能，系统能够自动预计算小时级、天级的统计数据，极大降低了实时报表查询的系统开销。此外，项目集成了 PostGIS 空间数据库引擎，能够高效存储监测点的经纬度信息，并支持复杂的空间查询，为系统的地图可视化功能提供了坚实的底层支持。

在应用开发层，后端采用基于 Python 的 FastAPI 框架，利用其异步编程特性以极低的资源消耗处理高并发的设备上报请求，并配合 SQLAlchemy ORM 框架保证代码的可维护性。前端则基于 Vue.js 3 构建单页面应用 (SPA)，结合 ECharts 图表库和 Leaflet 地图库，将枯燥的监测数据转化为直观的趋势图、热力图和仪表盘。在部署运维方面，系统全栈采用 Docker Compose 进行容器化编排，解决了环境一致性问题，使得系统易于在各种服务器环境中快速部署和迁移。综上所述，项目所选技术栈成熟稳定、文档完善且均为开源免费软件，完全具备技术实现的条件。

3.2 应用可行性

本系统的设计紧贴城市环境治理的实际需求，具有显著的社会效益和管理价值，能够有效解决当前噪声监管中的痛点。首先，系统满足了监管部门数字化转型的迫切需求。传统的噪声监管依赖人工巡查和事后投诉处理，效率低下且滞后，而本系统实现了全天候 7x24 小时无人值守监测。通过数据库触发器实现的自动告警机制，一旦噪声超标立即生成工单并通

知运维人员，将“被动响应”转变为“主动治理”，显著提升了环境执法的时效性。

其次，系统充分满足了公众的知情权与参与感。通过面向公众开放实时噪声地图和噪声排行榜，市民能够随时了解身边的声环境质量，增强了公众对环保工作的信任感和参与度。

最后，从经济效益与可扩展性角度来看，系统采用软硬件解耦设计，支持多种标准协议的 IoT 设备接入，降低了硬件采购门槛，基于开源软件构建也使得维护成本低廉。同时，系统架构设计灵活，未来可轻松扩展接入空气质量、温湿度等其他环境指标，打造综合性的智慧城市环境监测平台，具有极高的推广前景。

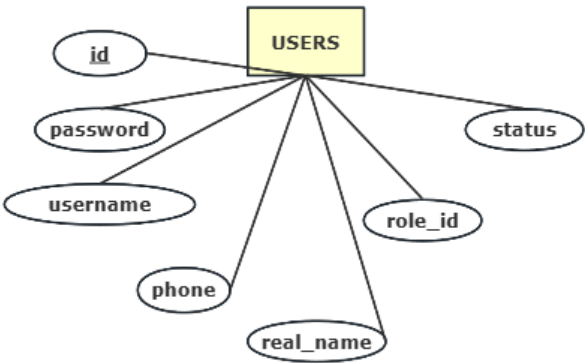
四. 概念设计

4.1 实体

1、用户 (User)

描述：系统的操作人员，包括管理员、运维员和普通用户。

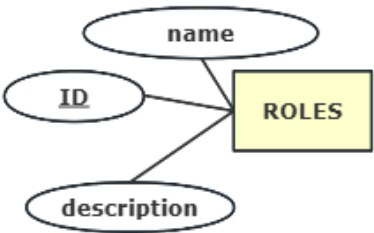
关键属性：用户 ID、用户名、密码哈希、真实姓名、联系电话、邮箱、所属角色 ID、账号状态（激活/锁定）、创建时间。



2、角色 (Role)

描述：定义用户的权限级别（如：超级管理员、区域运维员、公众用户）。

关键属性：角色 ID、角色名称、描述。

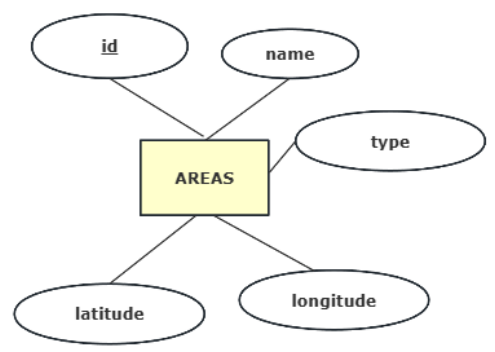


3、区域 (Area)

描述：城市中的地理功能区块，如居民区、工业区、商业区等。

关键属性：区域 ID、区域名称、区域类型（居住/商业/工业/交通/特殊）、中心经纬度、空间几何对象 (geom)。

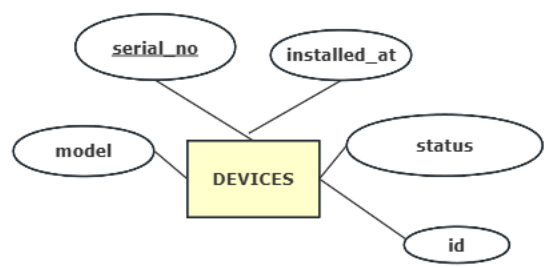
技术特点：使用 PostGIS geometry 类型存储空间数据。



4、设备 (Device)

描述：实际部署的物理硬件传感器。

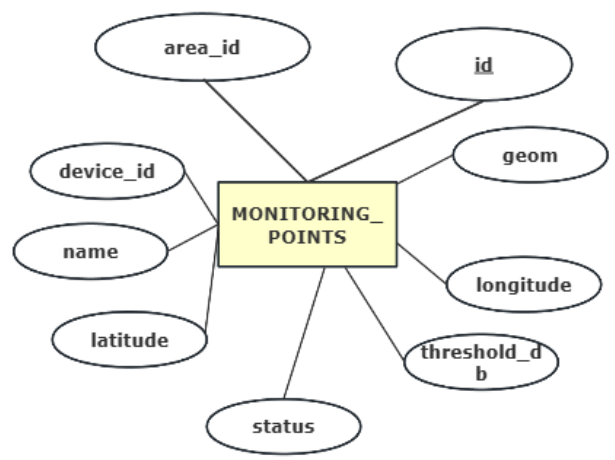
关键属性：设备 ID、序列号 (唯一标识)、型号、安装位置、设备状态（在线/离线/维护）。



5、监测点 (Monitoring Point)

描述：设备部署的具体逻辑站点，是数据采集的基本单元。

关键属性：站点 ID、站点名称、所属区域 ID、关联设备 ID、经纬度、空间几何对象 (geom)、默认阈值、状态。

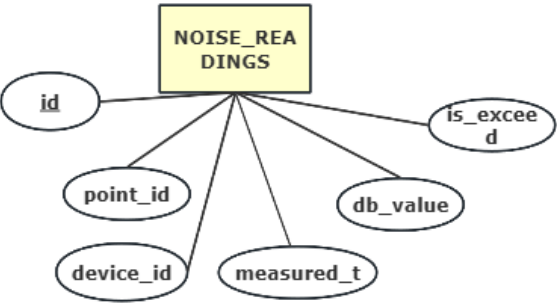


6、噪声数据 (Noise Reading)

描述：设备上报的实时监测数据。

关键属性：读数 ID、所属监测点 ID、测量时间 (measured_at)、噪声值 (dB)、温度、湿度、电池电量、是否超标标记。

技术特点：使用 TimescaleDB 超表 (Hypertable)，按时间自动分片，支持高吞吐写入。



7、告警 (Alert)

描述：当噪声值超过阈值或设备异常时生成的事件记录。

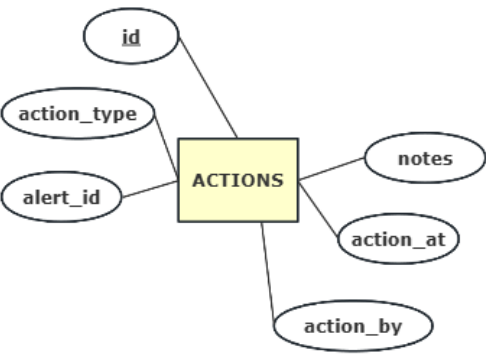
关键属性：告警 ID、关联读数 ID、所属监测点 ID、告警类型（阈值超标/设备异常/趋势异常）、严重程度、触发时间、触发时的噪声值、当时的阈值、当前状态（待处理/已确认/已解决）、解决时间、描述。



8、处理动作 (Action)

描述：记录运维人员对告警进行的操作历史（如确认告警、填写处理结果）。

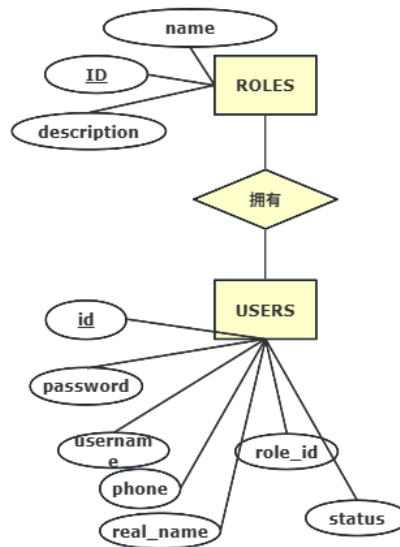
关键属性：动作 ID、关联告警 ID、动作类型、操作人 ID、操作时间、备注说明。



4.2 实体属性局部 E-R 图

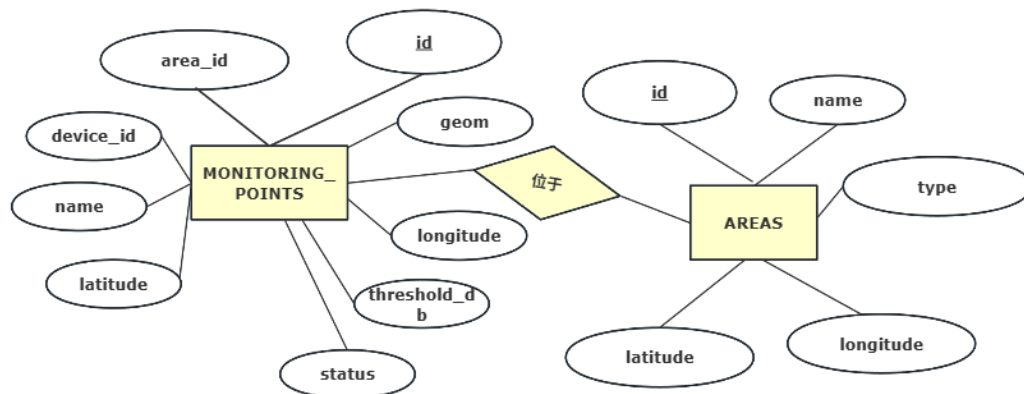
1、用户与角色的归属关系

用户实体与角色实体之间存在多对一的关系。一个角色（如“区域运维员”）可以被分配给多个不同的用户，但一个用户在系统中只能拥有一个主角色，用于确定其权限边界。



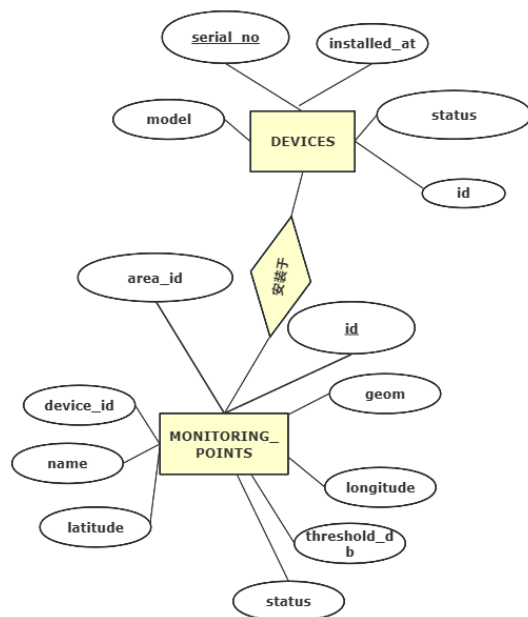
2、区域与监测点的包含关系

区域实体与监测点实体之间存在一对多的关系。一个功能区域内可以部署多个具体的监测点，但一个监测点在地理逻辑上只能归属于一个特定的区域。



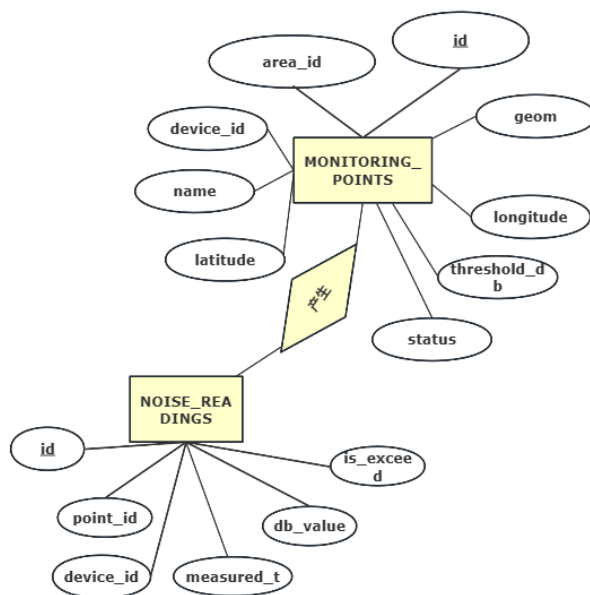
3、监测点与设备的部署关系

监测点实体与设备实体之间存在一对一（或多对一，视历史记录而定）的关系。在当前时刻，一个监测点只能安装一台物理设备进行采集；一台设备也只能被部署在一个监测点上。这是物理层面的绑定关系。



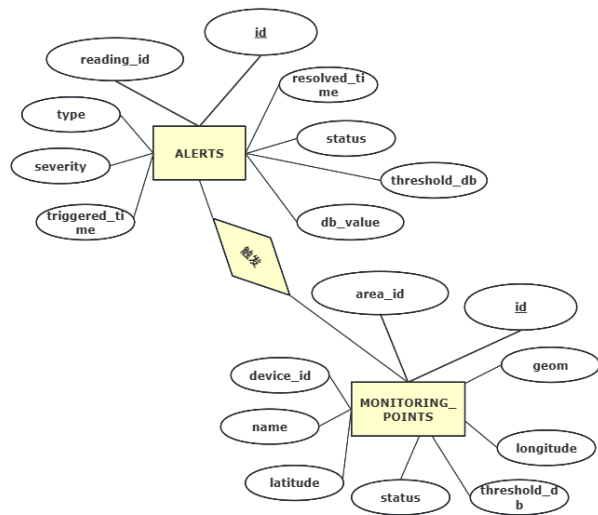
4、监测点与噪声数据的生成关系

监测点实体与噪声读数实体之间存在一对多的关系。一个监测点会随着时间推移，连续不断地生成海量的噪声监测数据（每分钟或每秒一条），这些数据共同构成了该站点的历史噪声曲线。



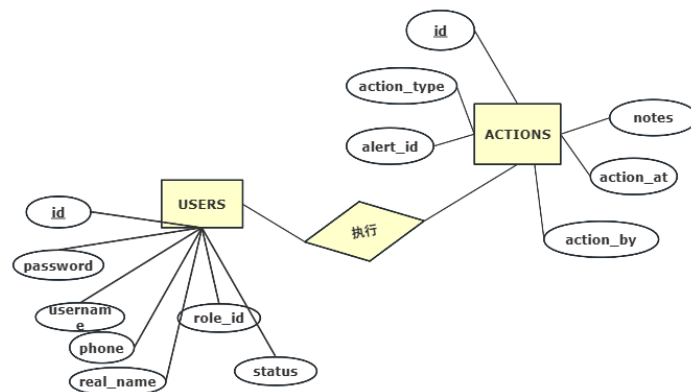
5、监测点与告警的触发关系

监测点实体与告警实体之间存在一对多的关系。当某个监测点的噪声值超过预设阈值或设备离线时，该站点会触发一条或多条告警记录。



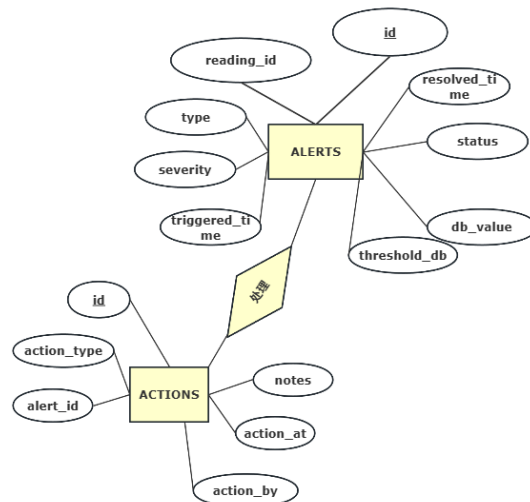
6、用户与处理动作的操作关系

用户实体与处理动作实体之间存在一对多的关系。一个运维人员或管理员可以对不同的告警执行多次处理操作，每一次操作都会被记录为一条带有操作人签名的动作记录。



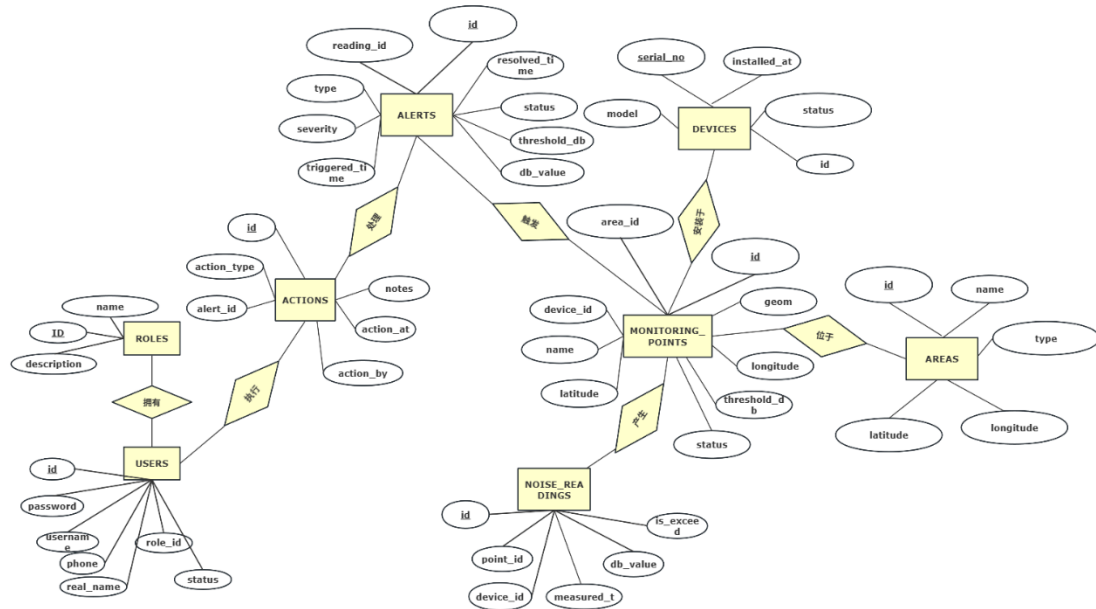
7、告警与处理动作的处置关系

告警实体与处理动作实体之间存在一对多的关系。一条告警记录在生命周期内，可能会经历多次处理动作，这些动作共同构成了告警的处理日志。



4.3 全局 E-R 图

本数据库使用 E-R 图构建数据库实体和关系，在 E-R 图中实体为用户、监测点、区域、噪声数据和报警记录等 8 个实体集。具体关系设计如下。



五. 逻辑设计

5.1 E-R 图向关系模型的转变

此处将描述关系进行转化得到关系模型，并对关系模型进行修改调整使其满足 3NF 原则。同时会将关系进行进一步简化去除冗余信息连接。

实体与关系转换：

1、角色 (roles)=(角色 ID, 角色名称, 角色描述)
(id, name, description)

2、用户 (users)=(用户 ID, 用户名, 密码哈希, 电子邮箱, 联系电话, 真实姓名, 角色 ID, 账号状态, 创建时间, 更新时间)
(id, username, password_hash, email, phone, real_name, role_id, status, created_at, updated_at)

3、区域 (areas)=(区域 ID, 区域名称, 区域类型, 中心纬度, 中心经度, 空间几何对象, 创建时间)
(id, name, type, latitude, longitude, geom, created_at)

4、设备 (devices)=(设备 ID, 设备序列号, 设备型号, 安装时间, 设备状态, 创建时间)

(id, serial_no, model, installed_at, status, created_at)

5、监测点 (monitoring_points)=(监测点 ID, 所属区域 ID, 关联设备 ID, 站点名称, 纬度, 经度, 空间几何对象, 默认阈值, 状态, 创建时间)

(id, area_id, device_id, name, latitude, longitude, geom, threshold_db, status, created_at)

6、噪声读数 (noise_readings)=(读数 ID, 所属监测点 ID, 采集设备 ID, 测量时间, 分贝值, 温度, 湿度, 电池电量, 是否超标, 创建时间)

(id, point_id, device_id, measured_at, db_value, temperature, humidity, battery_pct, is_exceed, created_at)

7、告警 (alerts)=(告警 ID, 关联读数 ID, 所属监测点 ID, 告警类型, 严重程度, 触发时间, 触发时分贝值, 触发时阈值, 处理状态, 解决时间, 描述, 创建时间)

(id, reading_id, point_id, type, severity, triggered_at, db_value, threshold_db, status, resolved_at, description, created_at)

8、处理动作 (actions)=(动作 ID, 关联告警 ID, 动作类型, 操作人 ID, 操作时间, 备注说明)

(id, alert_id, action_type, action_by, action_at, notes)

5.2 数据模型的优化及规范化设计

第三范式 (3NF) 的核心要求是在满足第二范式 (2NF) 的基础上, 消除非主属性对主键的传递依赖, 即每个非主属性都必须直接依赖于主键, 而不能依赖于其他非主属性。对本项目关系模式的分析表明, 绝大多数实体设计均严格遵循 3NF 原则。例如, 角色 (roles)、用户 (users)、设备 (devices)、处理动作 (actions) 等表, 其所有非主属性均直接描述主键对象, 不存在任何传递依赖关系。

然而, 在实际工程落地中, 为了满足高性能查询、历史数据溯源及空间计算的需求, 部分核心表进行了合理的反范式化设计。比如, 区域 (areas) 和 监测点 (monitoring_points) 表中同时存储了经纬度与 geom 空间对象, 虽然存在属性间的依赖, 但这符合 PostGIS 的标准设计模式, geom 字段是建立空间索引和进行高效空间计算(如查找附近的点)所必需的。

综上所述, 本项目的数据库设计在逻辑主体上遵循了 3NF, 但在核心的高频数据表和业务关键表中, 基于性能优化和业务完整性的考量, 进行了必要的反范式化处理。这种设计在工业界属于标准的最佳实践, 既保证了数据架构的清晰与一致性, 又有效兼顾了系统的运行效率与实际业务需求。

5.3 表结构设计

1、roles (角色表)

字段	类型	约束/默认值	键/引用
id	SERIAL	NOT NULL	PK

name	VARCHAR(50)	UNIQUE, NOT NULL	
description	VARCHAR(200)		

2、users（用户表）

字段	类型	约束/默认值	键/引用
id	SERIAL	NOT NULL	PK
username	VARCHAR(50)	UNIQUE, NOT NULL	
password_hash	VARCHAR(255)	NOT NULL	
email	VARCHAR(120)		
phone	VARCHAR(20)		
real_name	VARCHAR(50)		
role_id	INTEGER		FK → roles(id)
status	VARCHAR(20)	DEFAULT 'active'; CHECK ∈ {active,inactive,locked,pending}	
created_at	TIMESTAMPTZ	DEFAULT NOW()	
updated_at	TIMESTAMPTZ	DEFAULT NOW()	

3、areas（区域表）

字段	类型	约束/默认值	键/引用
id	SERIAL	NOT NULL	PK
name	VARCHAR(100)	NOT NULL	
type	VARCHAR(50)	CHECK ∈ {residential,commercial,industrial,traffic,special}	
latitude	DECIMAL(9,6)	CHECK: NULL 或 [-90, 90]	
longitude	DECIMAL(9,6)	CHECK: NULL 或 [-180, 180]	
geom	geometry(Point, 4326)	GENERATED ALWAYS AS（由经纬度生成） STORED	
created_at	TIMESTAMPTZ	DEFAULT NOW()	

4、devices（设备表）

字段	类型	约束/默认值	键/引用
id	SERIAL	NOT NULL	PK
serial_no	VARCHAR(100)	UNIQUE, NOT NULL	
model	VARCHAR(100)		
installed_at	TIMESTAMPTZ		
status	VARCHAR(20)	DEFAULT 'active'; CHECK ∈ {active,inactive,maintenance}	
created_at	TIMESTAMPTZ	DEFAULT NOW()	

5、monitoring_points（监测点表）

字段	类型	约束/默认值	键/引用
id	SERIAL	NOT NULL	PK
area_id	INTEGER	ON DELETE SET NULL	FK → areas(id)
device_id	INTEGER	ON DELETE SET NULL	FK → devices(id)
name	VARCHAR(100)	NOT NULL	
latitude	DECIMAL(9,6)	CHECK: NULL 或 [-90, 90]	
longitude	DECIMAL(9,6)	CHECK: NULL 或 [-180, 180]	
geom	geometry(Point, 4326)	GENERATED ALWAYS AS (由经纬度生成) STORED	
threshold_db	NUMERIC(5,2)	DEFAULT 65.00 ; CHECK ∈ [0, 200]	
status	VARCHAR(20)	DEFAULT 'active'; CHECK ∈ {active,inactive,maintenance}	
created_at	TIMESTAMPTZ	DEFAULT NOW()	

6、noise_readings（噪声表）

字段	类型	约束/默认值	键/引用
id	BIGSERIAL	NOT NULL	PK（复合）
point_id	INTEGER	NOT NULL ; ON DELETE CASCADE	FK → monitoring_points(id)
device_id	INTEGER		FK → devices(id)
measured_at	TIMESTAMPTZ	NOT NULL	PK（复合，时间列）
db_value	NUMERIC(5,2)	NOT NULL ; CHECK ∈ [0, 200]	
temperature	NUMERIC(4,1)		
humidity	NUMERIC(4,1)		
battery_pct	NUMERIC(4,1)		
is_exceed	BOOLEAN	DEFAULT FALSE	
created_at	TIMESTAMPTZ	DEFAULT NOW()	

7、alerts（告警表）

字段	类型	约束/默认值	键/引用
id	SERIAL	NOT NULL	PK
reading_id	BIGINT		FK→ noise_readings (id)

point_id	INTEGER	NOT NULL ; ON DELETE CASCADE	FK → monitoring_points(id)
type	VARCHAR(30)	DEFAULT 'threshold'; CHECK ∈ {threshold,anomaly,trend}	
severity	VARCHAR(20)	NOT NULL ; CHECK ∈ {low,medium,high,critical}	
triggered_at	TIMESTAMP TZ	NOT NULL	
db_value	NUMERIC(5,2)	NOT NULL	
threshold_db	NUMERIC(5,2)	NOT NULL	
status	VARCHAR(20)	DEFAULT 'open' ; CHECK ∈ {open,acknowledged,resolved,dissmissed}	
resolved_at	TIMESTAMP TZ		
description	TEXT		
created_at	TIMESTAMP TZ	DEFAULT NOW()	

8、actions（告警处理动作表）

字段	类型	约束/默认值	键/引用
id	SERIAL	NOT NULL	PK
alert_id	INTEGER	NOT NULL ; ON DELETE CASCADE	FK → alerts(id)
action_type	VARCHAR(50)	NOT NULL	
action_by	INTEGER	ON DELETE SET NULL	FK → users(id)
action_at	TIMESTAMPTZ	DEFAULT NOW()	
notes	TEXT		

5.4 物理设计

本系统的物理设计充分利用了 PostgreSQL 16 的高级特性以及 TimescaleDB 和 PostGIS 扩展，针对海量时序数据写入、空间地理查询和实时统计分析进行了深度优化。

5.4.1 存储结构与分区策略

针对核心的噪声监测数据表 `noise_readings`，系统采用了 TimescaleDB 的超表技术进行物理存储优化，以解决传统关系型数据库在亿级数据量下的性能瓶颈。

时间分区：

策略：将 `noise_readings` 表按 `measured_at` 时间字段自动切分为多个子表。

配置：设置分块时间间隔为 7 天。这意味着每周的数据会存储在一个独立的物理文件中，既保证了热数据的内存缓存命中率，又便于冷数据的归档管理。

列式压缩：

策略：启用 TimescaleDB 的列式压缩机制，将行存数据转换为列存数据。

配置：按 `point_id` 进行分段，并按时间排序。系统配置为自动压缩 30 天前的历史数据。

效果：压缩率通常可达 90% 以上，极大节省磁盘空间，同时提升历史统计查询的 I/O 效率。

数据保留：

策略：配置自动保留策略，仅保留最近 24 个月的原始监测数据，过期数据将由后台任务自动清理，无需人工干预。

5.4.2 索引设计

为了加速高频查询，系统在关键字段上建立了多种类型的索引。

表名	索引名称	索引类型	索引字段	应用场景
areas	idx_areas_geom	GIST	geom	空间查询：查找某坐标点所在的区域
monitoring_points	idx_points_geom	GIST	geom	空间查询：查找用户附近的监测点 (KNN)
noise_readings	idx_noise_readings_point_time	BTREE	point_id, measured_at DESC	核心查询：获取某站点最新的实时数据或历史曲线
noise_readings	idx_noise_readings_exceeded	BTREE	measured_at DESC (Partial)	过滤查询：仅查询 is_exceed=TRUE 的超标记录

表名	索引名称	索引类型	索引字段	应用场景
alerts	idx_alerts_status	BTREE	status, triggered_at DESC	运维查询：按状态筛选待处理的告警列表

5.4.3 视图与预计算

为了解决实时统计分析（如“查询某区域过去一个月的平均噪声”）带来的巨大计算开销，系统采用了 TimescaleDB 的连续聚合 (Continuous Aggregates) 技术。

每日监测点统计 (mv_daily_point_stats):

定义：按天 (1 day) 预计算每个监测点的样本数、平均值、最大/最小值及超标次数。

刷新策略：每小时自动增量刷新，计算最近 30 天的数据。

优势：将亿级数据的聚合查询降维到万级，查询响应时间从秒级降低到毫秒级。

每小时区域统计 (mv_hourly_area_stats):

定义：按小时 (1 hour) 预计算每个功能区域的整体噪声水平。

刷新策略：每小时自动刷新最近 7 天的数据。

实时地图视图 (v_points_map):

定义：这是一个普通视图，通过 DISTINCT ON 语法关联监测点表与最新的噪声读数，为前端地图展示提供“站点+最新状态”的宽表数据。

5.4.4 触发器与自动化

系统利用数据库触发器实现了核心业务逻辑的自动化，确保数据的一致性和实时性。

超标计算触发器 (trg_noise_exceed_compute):

时机：BEFORE INSERT (写入噪声数据前)。

逻辑：自动对比当前分贝值与站点阈值，设置 is_exceed 字段标记。

自动告警触发器 (trg_noise_alert_after_insert):

时机：AFTER INSERT (写入噪声数据后)。

条件：仅当 NEW.is_exceed = TRUE 时触发。

逻辑：自动向 alerts 表插入一条新告警记录，实现“监测即告警”的零延迟响应。

5.4.5 空间数据存储

几何类型：使用 PostGIS 的 `geometry(Point, 4326)` 类型存储经纬度坐标（WGS84 坐标系）。

生成列：利用 PostgreSQL 的 `GENERATED ALWAYS AS ... STORED` 特性，自动根据 `latitude` 和 `longitude` 字段生成 `geom` 对象，确保了业务字段与空间对象的一致性，无需应用层手动维护。

六. 项目管理

6.1 框架选择

前端框架：Vue.js 3 + Vite + Pinia + ECharts + Leaflet

后端框架：FastAPI + SQLAlchemy + Pydantic

数据库：PostgreSQL 16 + TimescaleDB + PostGIS

6.2 开发平台

操作系统：Windows

开发工具 (IDE)：Visual Studio Code

容器化平台：Docker & Docker Compose（用于统一编排数据库、后端与前端服务）

版本控制：Git

七. 系统实现

7.1 系统架构搭建

本系统采用经典的 B/S (Browser/Server) 架构，基于前后端分离的设计模式构建，整体架构自下而上分为数据感知层、数据存储层、业务逻辑层和应用表现层四个层次，各层之间通过标准协议进行交互，实现了高内聚、低耦合的系统特性。

最底层的数据感知层由分布在城市各功能区的噪声监测终端组成，设备实时上报环境噪声等原始数据。数据存储层采用 PostgreSQL 作为核心数据库引擎，并集成了 TimescaleDB 和 PostGIS 扩展。系统利用 TimescaleDB 的超表技术存储海量时序监测数据，并启用列式压缩以节省空间；利用 PostGIS 存储监测点和功能区的空间几何信息，支持高效的地理空间查询；同时利用物化视图自动预计算小时级和天级的统计报表，加速查询响应。

中间的业务逻辑层基于 Python FastAPI 框架构建高性能 RESTful API 服务。其中，数

据接入服务负责异步接收并清洗设备上报的数据，并通过数据库触发器实时判断是否超标；告警服务监听超标事件，生成告警记录并分发给相关运维人员；统计分析服务提供多维度的噪声数据聚合分析接口；权限管理服务则基于 RBAC 模型实现用户身份认证与接口权限控制。

最上层的应用表现层是基于 Vue.js 3 构建的单页面应用，包括集成 Leaflet 地图的实时监控大屏，直观展示全市监测点分布及实时噪声热力图；提供设备管理、告警处置、用户权限配置等功能的运维管理后台；以及利用 ECharts 展示噪声趋势曲线、超标统计柱状图等分析图表的数据可视化报表。整个系统通过 Docker Compose 进行统一编排与部署，确保了开发、测试与生产环境的一致性。

7.2 系统逻辑设计

本系统的核心功能逻辑围绕“数据采集—实时监控—告警处置—统计分析”这一闭环业务流展开。首先，在数据采集环节，分布在城市各处的 IoT 监测设备定时采集环境噪声数据，并通过加密的 API 接口发送至后端服务。后端接收到数据后，首先进行格式校验与清洗，随后将其写入 TimescaleDB 时序数据库。在此过程中，数据库内部的触发器会立即介入，将当前噪声值与该监测点预设的昼夜阈值进行比对。

一旦检测到噪声超标，系统将自动触发告警逻辑。触发器会即时在告警表中生成一条状态为“待处理”的告警记录，并记录触发时的快照数据。告警生成后，系统通过 WebSocket 或消息推送机制通知区域运维员。运维员登录系统后，可在“告警管理”模块查看告警详情，并执行确认、填写处理结果或关闭告警等操作，系统会完整记录每一步操作的时间与人员，形成闭环的审计日志。

在统计分析方面，系统摒弃了传统的实时全量计算模式，转而采用“预计算”逻辑。利用数据库的连续聚合功能，系统后台会按小时和天自动计算各监测点和区域的平均噪声、最大值及超标率，并将结果存储在物化视图中。当前端用户请求查看“过去 30 天噪声趋势”或“区域噪声排行”时，API 接口直接从物化视图中读取预先计算好的结果，从而在毫秒级内完成响应，极大地提升了用户体验并降低了服务器负载。此外，系统还通过 RBAC 权限模型，严格控制不同角色对上述功能模块和数据范围的访问权限，确保了业务逻辑的安全运行。

7.3 具体功能编写

系统的开发过程遵循敏捷开发模式，自底向上依次完成了数据库层、后端服务层和前端应用层的构建。首先进行的是数据库层的开发，编写了 01_tables.sql 等初始化脚本，定义了包括用户、设备、监测点及噪声读数在内的核心实体表，并配置了 TimescaleDB 的超表策略与 PostGIS 的空间索引。在此阶段，还重点编写了 03_triggers.sql 中的触发器逻辑，实现了噪声超标自动检测与告警生成的数据库级自动化，确保了核心业务规则的强一致性。

随后进入后端服务层的开发，基于 FastAPI 框架搭建了 RESTful API 服务。Pydantic

定义了严格的数据交互模型，确保前后端通信的数据规范；使用 SQLAlchemy 实现了 ORM 映射，封装了对数据库的增删改查操作。重点实现了 `api/readings.py` 中的数据接收接口，使其能够异步处理高并发的设备上报请求；以及 `api/alerts.py` 和 `api/stats.py`，分别用于提供告警管理和统计分析的数据服务。同时，集成了 JWT 认证模块，为系统添加了安全的身份验证机制。

最后是前端应用层的开发，使用 Vue.js 3 配合 Vite 构建了单页面应用。开发过程中，首先搭建了基于 Pinia 的状态管理仓库，用于存储用户信息和全局配置；然后利用 Leaflet 组件开发了“实时监控地图”，实现了监测点在地图上的打点与状态即时更新；使用 ECharts 开发了“数据分析看板”，将后端返回的统计数据转化为直观的折线图和柱状图。在完成各模块的单元测试后，通过 Docker Compose 将前后端及数据库服务进行统一编排，完成了整个系统的集成与部署。

7.4 功能测试

1、用户权限与认证模块测试

注册与登录：

测试新用户注册流程。

非法信息：

城市噪声环境治理平台

智慧监测 · 科学治理 · 宁静生活

用户注册

创建您的账号

123

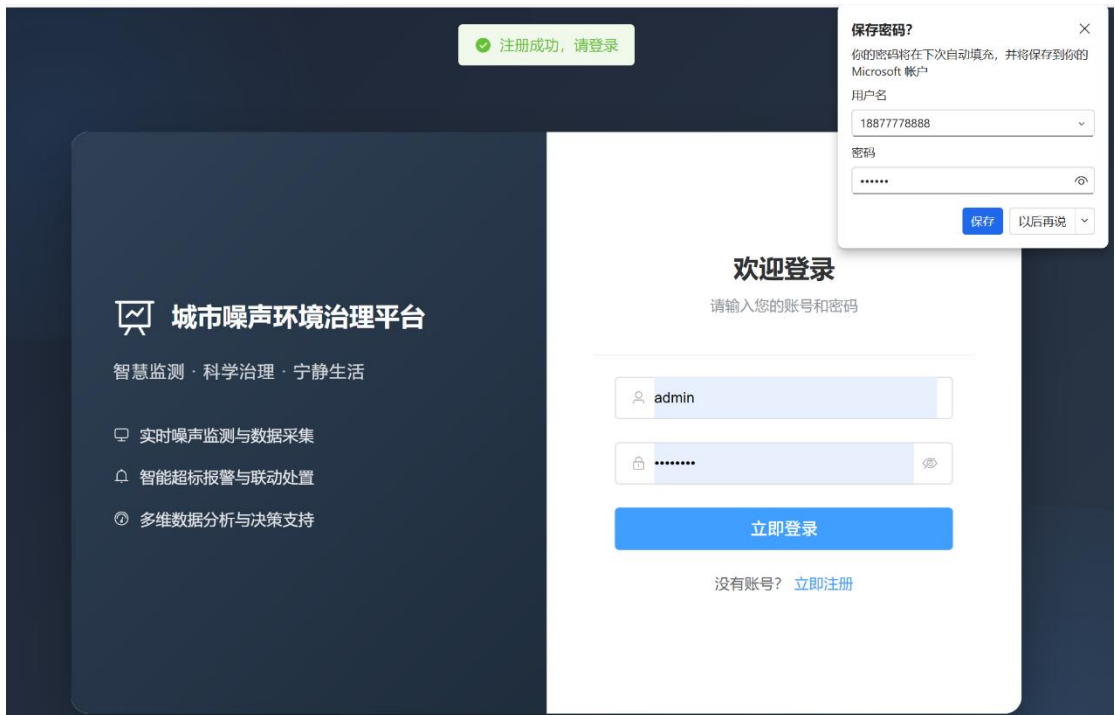
11111111111

请输入正确的手机号

123456

立即注册

已有账号？[去登录](#)



测试用户登录。
输入错误信息：



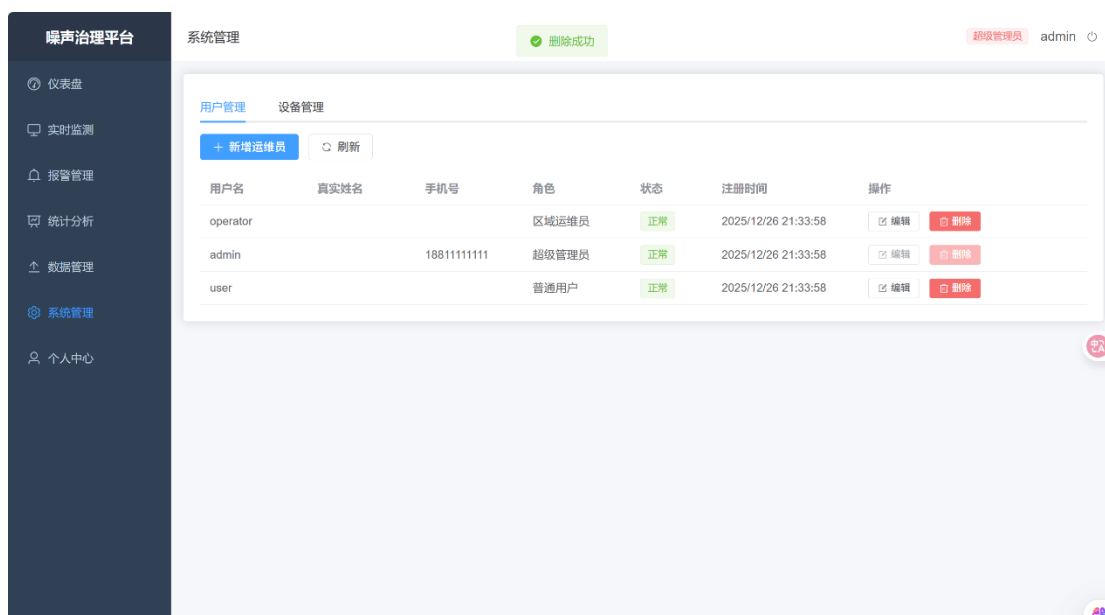
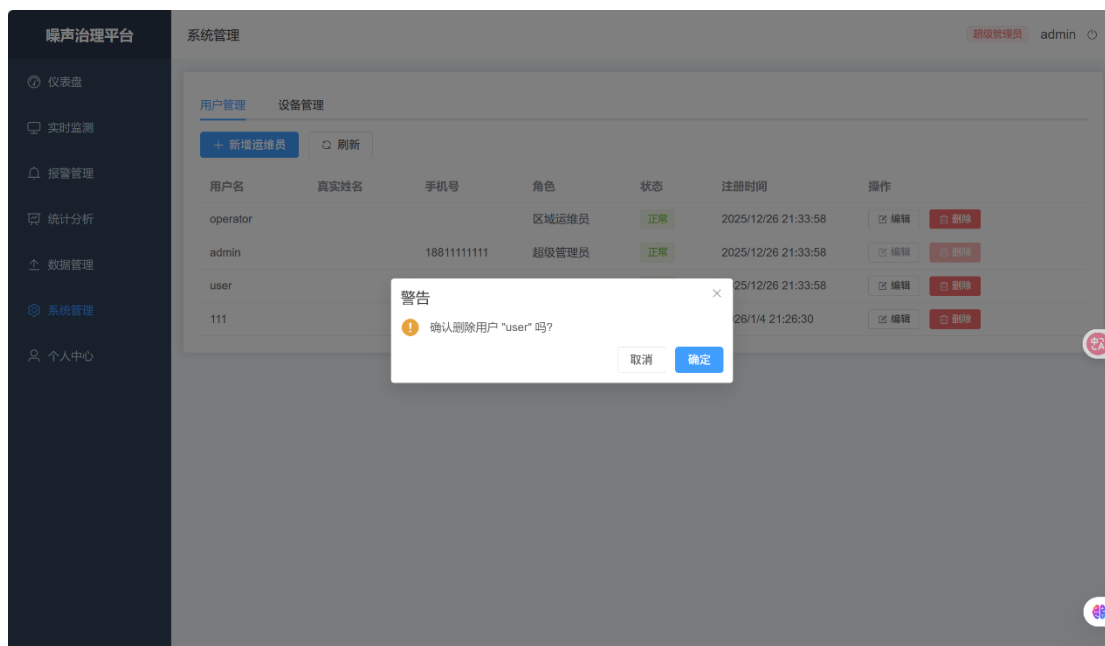
正确登录：



2、角色权限控制 (RBAC):

超级管理员：验证是否能访问所有模块（用户管理、设备管理、系统配置）。

删除用户：



更改用户身份：



区域运维员：验证是否看到区域的告警和设备，且不能删除用户或修改全局配置。

噪声治理平台

仪表盘

实时监测

报警管理

统计分析

数据管理

个人中心

报警管理

区域运维员operatc

搜索点位名称

区域类别

分页等级

处理状态

开始日期至结束日期

查询

刷新

报警时间	监测点位	区域类别	监测值(dB)	阈值	持续时间	状态	处置人	操作
2025-01-16T07:00:00Z	葵涌公园	special	69.00	60.00	344天23小时	已处置	123	定位
2025-01-15T20:00:00Z	万方广场	commercial	69.00	65.00	345天11小时	已处置	123	定位
2025-01-15T20:00:00Z	葵涌公园	special	71.00	60.00	待处理 353天	待处理	-	定位 处置
2025-01-15T20:00:00Z	大鹏广场	commercial	71.00	65.00	345天10小时	已处置	123	定位
2025-01-15T19:00:00Z	葵涌公园	special	74.00	60.00	待处理 353天	待处理	-	定位 处置
2025-01-15T19:00:00Z	大鹏广场	commercial	93.00	65.00	待处理 353天	待处理	-	定位 处置
2025-01-15T19:00:00Z	万方广场	commercial	70.00	65.00	待处理 353天	待处理	-	定位 处置
2025-01-15T15:00:00Z	葵涌公园	special	63.00	60.00	待处理 353天	待处理	-	定位 处置

无用户管理界面。

公众用户：验证是否只能访问公开的地图和排行榜，且无法进入后台管理界面。
只有基础功能，无管理用户、管理设备、导入数据等功能。

噪声治理平台

仪表盘

实时监测

报警管理

统计分析

个人中心

报警管理

普通用户111

搜索点位名称

区域类别

分页等级

处理状态

开始日期至结束日期

查询

刷新

报警时间	监测点位	区域类别	监测值(dB)	阈值	持续时间	状态	处置人	操作
2025-01-16T07:00:00Z	葵涌公园	special	69.00	60.00	344天23小时	已处置	123	定位
2025-01-15T20:00:00Z	万方广场	commercial	69.00	65.00	345天11小时	已处置	123	定位
2025-01-15T20:00:00Z	葵涌公园	special	71.00	60.00	待处理 353天	待处理	-	定位
2025-01-15T20:00:00Z	大鹏广场	commercial	71.00	65.00	345天10小时	已处置	123	定位
2025-01-15T19:00:00Z	葵涌公园	special	74.00	60.00	待处理 353天	待处理	-	定位
2025-01-15T19:00:00Z	大鹏广场	commercial	93.00	65.00	待处理 353天	待处理	-	定位
2025-01-15T19:00:00Z	万方广场	commercial	70.00	65.00	待处理 353天	待处理	-	定位
2025-01-15T15:00:00Z	葵涌公园	special	63.00	60.00	待处理 353天	待处理	-	定位

3、数据采集与处理模块测试

设备数据上报：

模拟发送格式错误、缺少必填字段或非法数值（如分贝值 <0）的数据，验证后端是否拒绝并报错。



超标自动检测：
发送一条低于阈值的噪声数据，验证 `is_exceed` 字段是否为 `false`，且未生成告警。
不生成告警记录，地图上显示为绿色（正常）。



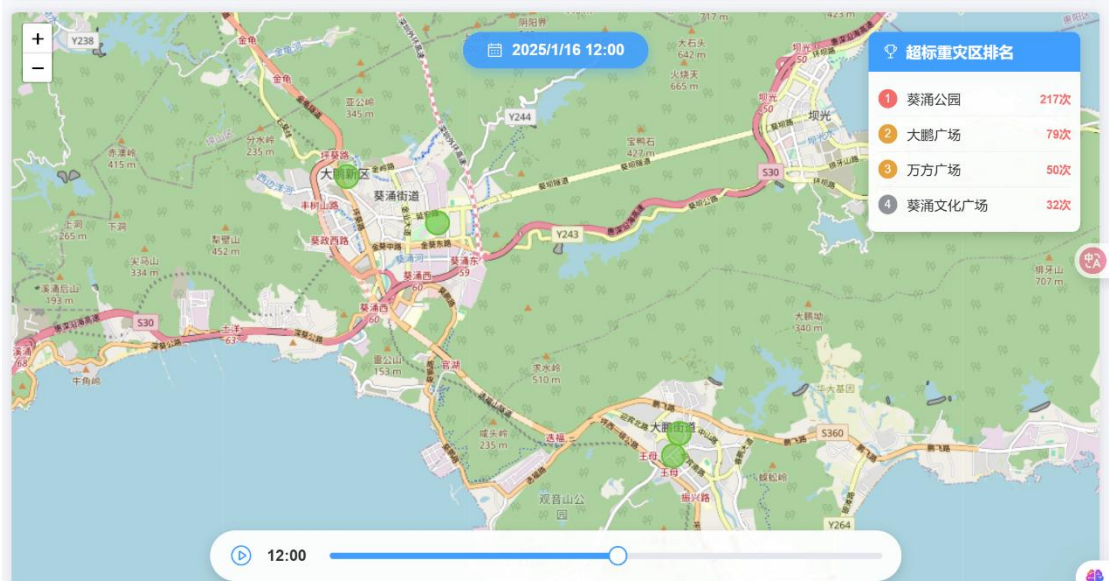
发送一条高于阈值的噪声数据，验证 `is_exceed` 字段是否为 `true`，且 `alerts` 表中是否自动生成了对应的新告警记录。

2025-01-15T20:00:00Z	大鹏广场	commercial	71.00	65.00	345天10小时	已处置	123	定位
2025-01-15T19:00:00Z	葵涌公园	special	74.00	60.00	待处理 353天	待处理	-	定位 处置
2025-01-15T19:00:00Z	大鹏广场	commercial	93.00	65.00	待处理 353天	待处理	-	定位 处置
2025-01-15T19:00:00Z	万方广场	commercial	70.00	65.00	待处理 353天	待处理	-	定位 处置
2025-01-15T15:00:00Z	葵涌公园	special	63.00	60.00	待处理 353天	待处理	-	定位 处置

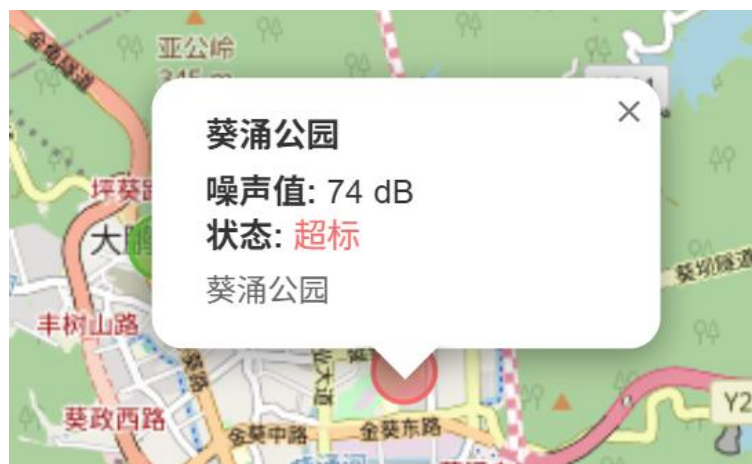
4、实时监控与可视化模块测试
数据看板展示：



实时地图展示:
验证地图上是否正确显示了所有监测点的位置图标。



验证点击图标后, 弹窗是否显示该站点最新的噪声值、状态。



验证当站点状态变为“超标”或“离线”时，地图图标颜色是否发生变化（如变红/变灰）。
统计图表渲染：



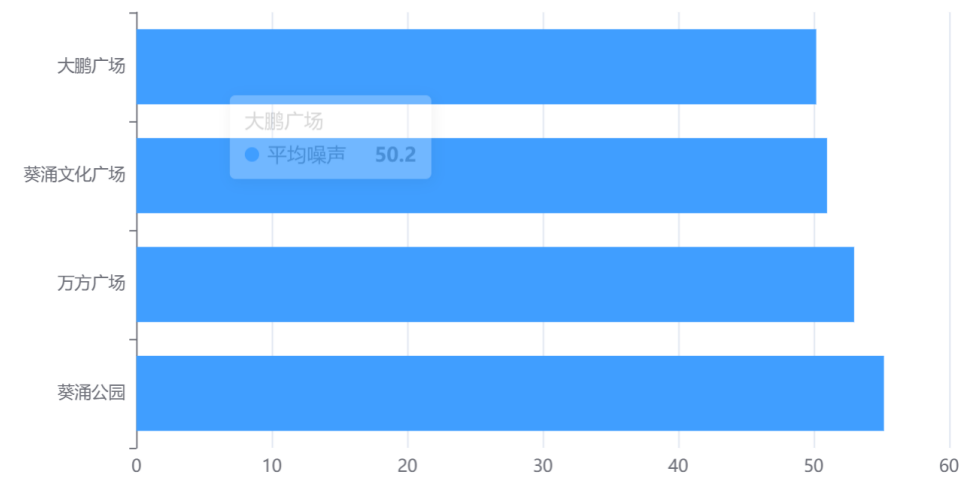
验证“24 小时趋势图”是否正确加载了历史数据。



验证“区域噪声排行”是否按分贝值从低到高正确排序。

趋势分析 区域对比

区域噪声排名 (Top 5)



5、告警管理模块测试

告警列表查询：

验证运维员能否看到待处理的告警列表。

报警管理 区域运维员 operator

搜索点位名称 区域类别 分页等级 处理状态 开始日期 至 结束日期 查询 刷新

报警时间	监测点位	区域类别	监测值(dB)	阈值	持续时间	状态	处置人	操作
2025-01-16T07:00:00Z	葵涌公园	special	69.00	60.00	344天23小时	已处置	123	定位
2025-01-15T20:00:00Z	万方广场	commercial	69.00	65.00	345天11小时	已处置	123	定位
2025-01-15T20:00:00Z	葵涌公园	special	71.00	60.00	待处理 353天	待处理	-	定位 处置
2025-01-15T20:00:00Z	大鹏广场	commercial	71.00	65.00	345天10小时	已处置	123	定位
2025-01-15T19:00:00Z	葵涌公园	special	74.00	60.00	待处理 353天	待处理	-	定位 处置
2025-01-15T19:00:00Z	大鹏广场	commercial	93.00	65.00	待处理 353天	待处理	-	定位 处置
2025-01-15T19:00:00Z	万方广场	commercial	70.00	65.00	待处理 353天	待处理	-	定位 处置

验证筛选功能（按时间、区域、状态筛选）是否有效。

报警管理

区域运维员operator

搜索点位名称

商业区

分页等级

处理状态

2024-01-09至2024-01-11

查询

刷新

报警时间	监测点位	区域类别	监测值(dB)	阈值	持续时间	状态	处置人	操作
2024-01-09T21:00:00Z	大鹏广场	commercial	70.00	65.00	待处理 725天	待处理	-	定位 处置
2024-01-09T21:00:00Z	葵涌文化广场	commercial	70.00	65.00	待处理 725天	待处理	-	定位 处置
2024-01-09T20:00:00Z	万方广场	commercial	66.00	65.00	待处理 725天	待处理	-	定位 处置
2024-01-08T21:00:00Z	大鹏广场	commercial	70.00	65.00	待处理 726天	待处理	-	定位 处置
2024-01-08T21:00:00Z	葵涌文化广场	commercial	69.00	65.00	待处理 726天	待处理	-	定位 处置
2024-01-08T20:00:00Z	大鹏广场	commercial	66.00	65.00	待处理 726天	待处理	-	定位 处置
2024-01-08T20:00:00Z	万方广场	commercial	68.00	65.00	待处理 726天	待处理	-	定位 处置

告警处置流程：
测试“确认告警”操作，验证状态是否变更为 acknowledged。

报警时间	监测点位	区域类别	监测值(dB)	阈值	持续时间	状态	处置人	操作
2025-01-16T07:00:00Z	葵涌公园	special	69.00	60.00	344天23小时	已处置	123	定位
2025-01-15T20:00:00Z	万方广场	commercial	69.00	65.00	345天11小时	已处置	123	定位

6、修改个人信息

噪声治理平台

个人中心

密码修改成功

个人信息

用户名admin邮箱admin@noise.local

角色超级管理员注册时间2025-12-26T13:33:58.351356Z

我的权限

用户管理修改密码区域管理监测点管理查看监测点数据导入数据清理报警管理统计查看

修改密码

原密码请输入原密码

新密码请输入新密码

确认密码请再次输入新密码

修改密码

八. 总结

在本次项目中，我设计并实现了一个功能完备的“城市噪声在线监测系统”，该系统不仅实现了对城市环境噪声数据的实时采集、存储与管理，还集成了实时监控大屏、超标自动告警、多维度统计分析以及基于 RBAC 的用户权限管理等核心功能。整体系统既满足了环保监管部门对噪声污染进行精细化治理的业务需求，也为公众提供了一个了解身边声环境质量的透明窗口。

在上学期的数据库设计中，我深入分析了物联网时序数据的特点，优化了实体关系模型，并针对性地设计了基于 TimescaleDB 的超表结构和 PostGIS 的空间索引，为本学期的高性能系统实现奠定了坚实的数据基础。在本学期的开发过程中，我严格遵循软件工程的生命周期规范，完整经历了从问题定义、可行性分析、需求分析到系统设计、编码实现及测试维护的全流程。在需求分析阶段，我通过绘制数据流图和编写数据字典，清晰界定了系统的数据边界与流转逻辑；在系统设计阶段，我采用了前后端分离的现代化架构，利用 Python FastAPI 高效处理并发请求，结合 Vue.js 3 与 ECharts、Leaflet 等前端库实现了丰富的数据可视化交互。特别是在物理设计环节，我通过配置数据库触发器实现了“监测即告警”的自动化逻辑，利用物化视图解决了海量数据统计的性能瓶颈，充分体现了数据库技术在解决实际工程问题中的核心价值。

通过本次项目的实践，我不仅熟练掌握了 PostgreSQL 高级特性及全栈开发技术，更深刻理解了如何将理论知识转化为解决复杂现实问题的工程能力。无论是系统架构的宏观把控，还是数据库性能优化的微观细节，我的技术视野与实践能力都得到了显著提升。希望在未来的学习与工作中，能够有机会继续深入探索数据库技术在智慧城市与物联网领域的更多创新应用。

参考文献

- [1] 殷丽萍, 王开德, 王剑敏. 基于生态环境治理预警系统的城市环境噪声污染监测技术的探讨[J]. 皮革制作与环保科技, 2022, 3(4): 74-76.
- [2] 谭博. 基于物联网和人工智能的城市噪声环境自动监测方法[J]. 自动化应用, 2025, 66(10): 20-22. DOI:10.19769/j.zdhy.2025.10.006.
- [3] 张洪浩, 黄春华, 刘杞元, 等. 高校校园声环境监测与评价——以南华大学红湘校区为例[J/OL]. 噪声与振动控制, 1-7[2025-06-08]. <http://kns.cnki.net/kcms/detail/31.1346.TB.20250414.1706.010.html>.
- [4] 杨意, 吴方英. 城市道路交通噪声监测与降噪措施研究[J]. 生态与资源, 2025, (02): 108-110. DOI:10.20260/j.cnki.styzy.2025.02.036.
- [5] 张维. 民“声”大事治理有方[N]. 法治日报, 2025-02-21(005). DOI:10.28241/n.cnki.nfzrb.2025.000993.
- [6] 毛玉如, 汪赟, 张建勋. 中国噪声自动监测的现状和发展[J]. 科技导报, 2024, 42(20): 23-31.